

# am5061.py — reference

Every class and function in the shared course module.

**What this is.** `am5061.py` is the shared helper module every case notebook uses. It is deliberately thin: it wraps CoolProp with the notation used in lectures, adds an Excel writer, and supplies the exchanger relations. It does **not** hide the engineering — every case still writes its own equations.

The module is written into each notebook by a setup cell, so it is always present and always the same version. Import it as `am`:

```
import am5061 as am
```

## 1 Temperature

| CALL                      | DOES                                   |
|---------------------------|----------------------------------------|
| <code>am.K(30)</code>     | Celsius → kelvin. Use at the boundary. |
| <code>am.C(303.15)</code> | Kelvin → Celsius. Reporting only.      |

Never write `+ 273.15` inside a formula. Convert once, on the way in, and once on the way out. Most unit errors in this course come from mixed scales inside an expression.

## 2 State — a thermodynamic state

Construct with **exactly two** independent properties. That is the phase rule, not a quirk of the code, and the class refuses anything else.

```
st = am.State("R134a", P=1e6, T=am.K(70))
st = am.State("Water", P=101325, Q=0)      # saturated liquid
st = am.State("R134a", P=p_cond, H=h2)    # from an enthalpy
```

Read properties as attributes. The names match the lecture notation rather than CoolProp's letter codes, so you do not need the CoolProp manual open:

| ATTRIBUTE              | QUANTITY                            | UNITS             |
|------------------------|-------------------------------------|-------------------|
| <code>.T / .T_C</code> | temperature                         | K / °C            |
| <code>.p</code>        | pressure                            | Pa                |
| <code>.h</code>        | specific enthalpy                   | J/kg              |
| <code>.s</code>        | specific entropy                    | J/kg·K            |
| <code>.d</code>        | density                             | kg/m <sup>3</sup> |
| <code>.cp</code>       | specific heat at constant pressure  | J/kg·K            |
| <code>.mu</code>       | dynamic viscosity                   | Pa·s              |
| <code>.k</code>        | thermal conductivity                | W/m·K             |
| <code>.x</code>        | vapour quality (–1 if single phase) | –                 |
| <code>.phase</code>    | phase name from CoolProp            | –                 |

**Quality is –1, not an error, when the state is single phase.** Always test `0 <= st.x <= 1` before you use it. And note CoolProp calls a point exactly on the dome `twophase`, which reads oddly for a saturated vapour but is correct.

### 3 Saturation

```
am.sat_liquid("Water", p=45e5)      # State on the bubble line
am.sat_vapour("Water", T=am.K(100))
am.p_sat("R134a", am.K(45))         # Pa
am.T_sat("R134a", 1e6)              # K
am.h_fg("Water", p=45e5)            # latent heat, J/kg
am.glide("R407C", 1e6)              # dew minus bubble, K
am.critical("CO2")                  # {'T':..., 'p':...}
am.fluids()                         # all 136 names
```

**Blends have no single saturation temperature.** `T_sat` returns the **bubble point**. For R407C, R410A or any zeotropic mixture, use `glide()` to see the spread and quote both ends. Asking for "the" saturation temperature of a blend is the commonest error in Week 11 and 12.

### 4 `solve` — find where a function is zero

```
x = am.solve(f, x0)                  # secant from a guess
x = am.solve(f, x0, bracket=(a, b)) # Brent, always converges
```

Give a bracket whenever you can. Brent's method cannot fail if the bracket is valid, whereas a secant from a bad guess can wander anywhere. If the bracket is invalid you get a message naming both end values — that message is usually telling you the physics is not where you thought.

```
ValueError: f(1)=2.4 and f(900)=18.1 have the same sign, so no root
is bracketed. Widen the bracket or check the equation.
```

### 5 `sweep` — run a parameter study

The swept function must return a **dict**. The result drops straight into `to_excel`.

```
rows = am.sweep(lambda t: cycle(T_cond=am.K(t)),
                 np.arange(50, 81, 2.5), name="T_cond, C")
```

## 6 Exchanger relations

```
am.lmtD(dT1, dT2)           # log-mean; arithmetic when the ends are close
am.effectiveness("counter", NTU, Cr)  # rating direction
am.ntu_required("counter", eps, Cr)   # design direction
```

Configurations: `counter`, `parallel`, `shell1`, `cross-both-unmixed`, `cross-Cmax-mixed`, `cross-Cmin-mixed`.

**Cr = 0 is the phase-change limit** and every configuration collapses to  $1 - \exp(-NTU)$ . That is why boilers and condensers are easy: the configuration question disappears. Week 8 is built on this.

`ntu_required` tells you when your target is impossible rather than returning a silly number:

```
ValueError: effectiveness 0.95 is unreachable for parallel at Cr=0.8;
the limit as NTU->infinity is 0.555556.
```

## 7 exergy

```
am.exergy(st)           # J/kg, Chennai dead state by default
am.exergy(st, T0=am.K(15), p0=101325)
```

Flow exergy is  $(h - h_0) - T_0(s - s_0)$ . The dead state defaults to 30 °C and 101.325 kPa — Chennai ambient.

**The dead state is a design choice, not a constant.** The same plant analysed against a Chennai ambient and a European one gives different exergy destruction. Always report which you used.

## 8 to\_excel — the deliverable

```
am.to_excel("D6_Condenser.xlsx",
            {"Zone summary": zone_rows, "Profile": rows},
            title="AM5061 D-6 . Condenser design",
            summary=[("Duty", 10e3, "W"), ("Length", L, "m")],
            sources=[("R134a properties", "CoolProp 8.0.0"),
                    ("Condensation", "Shah (1979)"]])
```

Every sheet gets AutoFilter, a frozen header row, and columns sized to content. Numbers are written as **numbers**, not text, so the filters and formulas actually work. A Summary sheet and a Sources sheet are always produced.

**Fill in sources**. It is not decoration. A result whose provenance nobody can trace is not an engineering deliverable, and it is where marks are lost most often.

## 9 Plot style

```
am.style_plots()
```

Call once near the top. Figures then match the lecture decks, so a plot pasted into your report looks like it belongs there. Colours are available as `am.NAVY`, `am.ORANGE`, `am.BLUE`, `am.MUTED`.